Lumora Labs RAG Design Guide

RAG Design Guide - version 2.1 - effective 2026-06-10. Owner: Retrieval squad (tech lead: Mateo Lindqvist). Applies to every retrieval-augmented feature built by the Lumora Labs AI Engineering team.

1. What retrieval-augmented generation is

Retrieval-augmented generation (RAG) answers a question in two stages. First, a retriever searches a document collection for the passages most relevant to the question. Second, a language model generates an answer using those passages as context. The model's parametric knowledge (what it learned during training) is complemented by non-parametric knowledge (the documents), which can be updated at any time without retraining.

RAG addresses three weaknesses of a plain LLM: it does not know private or recent information, it cannot show where an answer came from, and it tends to produce plausible but unsupported statements. A well-built RAG system answers from the documents, cites the passages it used, and says that it does not know when the documents do not contain the answer.

A RAG system has an offline indexing pipeline and an online query pipeline. The indexing pipeline loads documents, splits them into chunks, computes an embedding for each chunk, and stores chunks and vectors. The query pipeline embeds the question, retrieves candidate chunks, optionally reranks them, assembles a prompt, generates an answer, and validates citations. Most quality problems in production RAG systems are retrieval problems, not generation problems: if the right passage is not retrieved, no prompt can fix the answer.

This guide describes each stage, the trade-offs involved, and the defaults the team has agreed on. Defaults are collected in the table in section 11. Deviating from a default is allowed when an evaluation report shows the change improves the release gate metrics defined in the LLM Evaluation Playbook.

2. Document ingestion and parsing

Retrieval quality is bounded by parsing quality. Each file format needs its own loader. Plain text and Markdown can be read directly. Word documents should be read in document order, including tables, so that a table stays next to the paragraph that introduces it. PDFs are the hardest format: a PDF stores positioned glyphs, not paragraphs, and text extraction may merge columns, drop table structure or insert hyphenation and headers on every page. Scanned PDFs require OCR before any of this applies.

After extraction, text is normalised: line endings are unified, trailing whitespace is removed and runs of blank lines are collapsed. Normalisation must happen before chunking, and the normalised text must be stored, because chunk character offsets and evaluation evidence spans refer to the normalised text and not to the original file.

Every document is stored with a content hash, computed on the normalised text. When the indexing pipeline runs again, unchanged documents (same hash) are skipped. A changed document has all of its old chunks removed from the index before the new chunks are added; updating only the chunks that differ is not worth the complexity. A document whose source file has been deleted must be removed from the index, otherwise the assistant keeps citing a policy that no longer exists.

Metadata captured at ingestion time includes the source path, document type, title, version and effective date where available, and the ingestion timestamp. Metadata is used for filtering (for example, only current policies) and for showing citations to users.

3. Chunking

Documents are split into chunks because embedding models have input limits, because a single vector cannot represent a long document well, and because the generator should receive only the relevant parts of a document. Chunk size is a trade-off. Small chunks give precise matches but lose surrounding context; large chunks keep context but dilute the embedding with unrelated content and consume more of the context window.

Fixed-size chunking cuts text every N characters or tokens. It is simple but cuts through sentences and tables. Recursive chunking tries a list of separators in order, for example paragraph breaks, then line breaks, then sentence ends, then spaces, and only falls back to the next separator when a piece is still too large. It keeps paragraphs intact whenever they fit. Structure-aware chunking uses the document's own structure, such as Markdown headings or DOCX heading styles, to split at section boundaries and attaches the section heading to each chunk. Semantic chunking places boundaries where the embedding similarity between consecutive sentences drops; it can produce better boundaries but costs embedding calls at indexing time and is harder to reproduce.

Overlap repeats the end of one chunk at the start of the next, so that a fact that falls on a boundary is fully contained in at least one chunk. Overlap increases index size and can return near-duplicate chunks; the team default is 100 characters, which is 12.5% of the default chunk size of 800 characters.

Tables need special care. A table row that is separated from its header row loses its meaning: the row 'G4 | 35' means nothing without the column names. Team rule: tables must not be split in the middle of a row, and when a table is longer than one chunk, every chunk that contains table rows repeats the header row.

Every chunk stores the character offsets of its start and end in the normalised document text and a hash of its text. Offsets allow citations to point to an exact span and allow evaluation to decide whether a retrieved chunk overlaps the evidence for a question, independently of how the document was chunked.

4. Embeddings

An embedding model maps a text to a dense vector of fixed dimension, typically a few hundred to a few thousand floating-point numbers. The model is trained so that texts with similar meaning have vectors that point in similar directions. Similarity is usually measured with cosine similarity, the cosine of the angle between two vectors. If vectors are normalised to unit length, cosine similarity equals the inner (dot) product, which vector indexes compute efficiently. Team rule: all vectors are L2-normalised before they are stored.

Many embedding models are asymmetric: they expect a different task setting, or a different prefix, for queries and for documents, because a short question and a long passage that answers it are different kinds of text. Using the document setting for queries usually lowers retrieval quality. Check the model card and use the query setting at query time.

Absolute similarity scores are not comparable across embedding models. Some models produce scores spread between 0 and 1, others compress almost all scores into a narrow band, so that unrelated passages score 0.6 and relevant ones 0.7. A similarity threshold for abstention must therefore be calibrated on the golden set for the specific model, never copied from another system.

Some models support reduced output dimensions. Smaller vectors reduce memory and search time at a small cost in quality. The team default is 768 dimensions for production indexes; experiments may use the model's full dimension.

Embeddings are cached in the source database, keyed by the hash of the chunk text and the embedding model name. Re-indexing a document whose chunks did not change therefore costs no embedding calls. When the embedding model changes, every vector must be recomputed: vectors from two different models are not comparable and must never be mixed in one index.

5. Vector stores and indexes

A vector index answers the question: which stored vectors are closest to this query vector? An exact (flat) index compares the query with every vector. It is simple, has perfect recall and is fast enough for up to roughly a million vectors on a single machine. Approximate nearest neighbour (ANN) indexes trade a little recall for much faster search on large collections.

HNSW (hierarchical navigable small world) builds a layered proximity graph and searches it greedily from the top layer down. Its main parameters are M, the number of links per node, which controls memory and graph quality; efConstruction, the candidate list size during building; and efSearch, the candidate list size at query time, which trades latency for recall. IVF (inverted file) indexes cluster the vectors and search only the clusters nearest to the query; the nprobe parameter sets how many clusters are searched. Product quantisation (PQ) compresses vectors into short codes to reduce memory, at a further cost in accuracy.

Team rule: the relational database is the source of truth for documents, chunks, offsets and cached embeddings. The vector index is a derived artefact keyed by chunk id and can be rebuilt from the database at any time. When the index and the database disagree, the index is rebuilt (error code LUM-1002 in the LLMOps Incident Runbook). Index ids must never be reused for different chunks.

Metadata filtering restricts search to chunks matching conditions such as document type or effective date. Filtering after search can return fewer than K results; filtering before or during search is preferred when the vector store supports it.

6. Hybrid search

Dense retrieval matches meaning but can miss exact terms: product codes, error codes such as LUM-2001, names, and rare acronyms. Lexical retrieval with BM25 matches exact terms well but misses paraphrases. BM25 scores a document by the frequency of the query terms in it, weighted by how rare each term is across the collection (inverse document frequency) and normalised by document length. Its parameter k1, usually between 1.2 and 2.0, controls term-frequency saturation, and b, usually 0.75, controls length normalisation.

Hybrid search runs both retrievers and merges their result lists. Because BM25 scores and cosine similarities are on different scales, the team merges by rank, not by score, using reciprocal rank fusion (RRF). Each document receives the sum over retrievers of 1 / (k + rank), where rank is its position in that retriever's list and k is a constant, conventionally 60, that reduces the influence of the very top ranks. RRF needs no score normalisation and no tuning of weights, which makes it robust.

Team default: hybrid search is enabled for every production index. Each retriever returns 50 candidates, the lists are fused with RRF using k = 60, and the fused list is passed to the reranker. Hybrid search is particularly important for corpora that contain codes, identifiers and tables.

7. Reranking

Embedding models are bi-encoders: the query and each document are encoded separately, which makes search fast because document vectors are computed once, but limits accuracy because the model never sees query and document together. A cross-encoder reranker takes the query and one candidate passage as a single input and outputs a relevance score. It is far more accurate but far too slow to apply to a whole collection, so it is applied only to the candidates returned by the first stage.

Team default: rerank the top 50 fused candidates with a cross-encoder and keep the top 5 for the prompt. An LLM can also be used as a reranker or as a relevance grader that labels each candidate as relevant or not; this is more expensive and is used in the agentic query pipeline, not in the fast pipeline.

Reranking changes the score distribution. When a reranker is used, the abstention threshold must be calibrated on reranker scores, not on embedding similarities.

8. Context assembly, citations and abstention

The selected chunks are numbered and placed in the prompt with their source, inside clear delimiters. The instructions tell the model to answer only from the passages, to cite the passage number for every claim, and to state that the answer is not available when the passages do not contain it. Retrieved passages are untrusted content: instructions that appear inside a document must be ignored.

Citations are validated in code after generation. A citation that refers to a passage number that was not supplied is removed. An answer that claims to be answerable but contains no valid citation is replaced by an abstention and logged with error code LUM-3001.

Abstention has two layers. Before generation, if no retrieved chunk reaches the calibrated similarity threshold, the system abstains without calling the model (logged as LUM-1001). After generation, the model's structured output includes an answerable field, and the system abstains when the model reports that the passages are insufficient. An honest 'I don't know' is always better than a confident wrong answer; abstention accuracy is a release gate metric.

The total context passed to the generator is capped at 6,000 tokens. Ordering matters less with five short passages than with long contexts, but the most relevant passage is placed first.

9. Query transformation

Users write short, ambiguous or conversational questions. Query transformation rewrites the question before retrieval to improve recall. Query rewriting asks a model to produce a clearer, self-contained search query, for example by resolving pronouns from the conversation history ('what about its overlap?' becomes 'what is the default chunk overlap?'). Multi-query retrieval generates several paraphrases, retrieves for each, and fuses the results, which helps when the user's wording differs from the documents' wording.

Query decomposition splits a compound question into sub-questions that are retrieved separately, for example a question that compares two policies. HyDE (hypothetical document embeddings) asks the model to write a hypothetical answer and embeds that answer instead of the question, on the idea that an answer-shaped text is closer in embedding space to the real answer passage than the question is. HyDE can help on short queries but can also pull retrieval towards the model's own hallucinated content.

Every transformation adds at least one model call of latency and cost, and a bad rewrite can make retrieval worse. Team rule: query transformation is not used in the fast pipeline. In the agentic pipeline, the query is rewritten only when the relevance grader finds no relevant passage, and at most 2 rewrites are attempted before the system abstains.

The original user question, not the rewritten query, is always passed to the generator and recorded in the evaluation log, so that answers are judged against what the user actually asked.

10. Operating and maintaining indexes

An index is not built once. Documents are added, changed and retired, and the indexing pipeline must keep the index consistent with the corpus. Incremental indexing relies on the content hash of each document and the embedding cache keyed by chunk-text hash, so that a typical nightly run re-embeds only the chunks that actually changed. A full rebuild is needed only when the embedding model or the chunking configuration changes; in that case the new index is built side by side and switched over after its evaluation run passes the release gates.

Each document stores the chunking configuration it was split with. If the configured chunk size or overlap differs from the stored one, the document is treated as changed and is re-chunked, even when its content is identical. This prevents an index that silently mixes chunks produced by different configurations.

The ML Platform group monitors the following signals for every production index: the number of documents and chunks, the age of the oldest unprocessed document change, the embedding cache hit rate during indexing, the rate of LUM-1001 abstentions at query time, and the rate of LUM-1002 index consistency errors. A sudden rise in LUM-1001 abstentions after an indexing run usually means documents failed to parse, for example a PDF that lost its text layer.

Retired documents must be removed from the index on the same day they are withdrawn. Superseded policy versions are either deleted or tagged with an end date and excluded by a metadata filter, so that the assistant never answers from an outdated version when a newer one exists.

Backups cover the source database only. Because the vector index is a derived artefact, it is rebuilt from the restored database rather than backed up separately; restoring an index file from a different point in time than the database is a common cause of LUM-1002 errors.

11. Recommended defaults

The following defaults apply to every new RAG feature. They are the starting point for experiments run by the retrieval optimisation process; changes are accepted only with an evaluation report.

Parameter	Team default	Notes
Chunking strategy	Recursive, structure-aware	Split at headings first
Chunk size	800 characters	About 150-200 tokens of English text
Chunk overlap	100 characters	12.5% of chunk size
Tables	Never split mid-row	Repeat header row in every chunk
Embedding dimension	768	Full dimension allowed in experiments
Similarity	Cosine	Vectors L2-normalised, inner-product index
Vector index	Flat (exact)	HNSW above 1 million chunks
HNSW parameters	M = 32, efSearch = 128	Only when HNSW is used
Hybrid search	Enabled	Dense + BM25, 50 candidates each
Fusion	RRF, k = 60	Rank-based, no score normalisation
Reranker	Cross-encoder on top 50	Keeps top 5
Final top_k	5	Passages sent to the generator
Abstention threshold	Calibrated per model	Never copied between models
Max context	6,000 tokens	Retrieved passages only
Generator temperature	0.2	As in the AI Engineering Handbook

12. Anti-patterns and review checklist

- Tuning prompts before measuring retrieval. Check Recall@5 first.
- Copying a similarity threshold from another embedding model.
- Mixing vectors from two embedding models in one index.
- Treating the vector index as the source of truth and losing chunk offsets.
- Splitting tables so that rows lose their headers.
- Evaluating with chunk ids as relevance labels; labels must be evidence spans so that metrics stay comparable when chunking changes.
- Letting instructions found inside retrieved documents change the system's behaviour.
- Shipping without abstention: a RAG system that always answers will hallucinate on out-of-scope questions.

Before a RAG feature is released, the Retrieval squad tech lead reviews the configuration against this guide, and the evaluation report must pass every blocking release gate in the LLM Evaluation Playbook.